

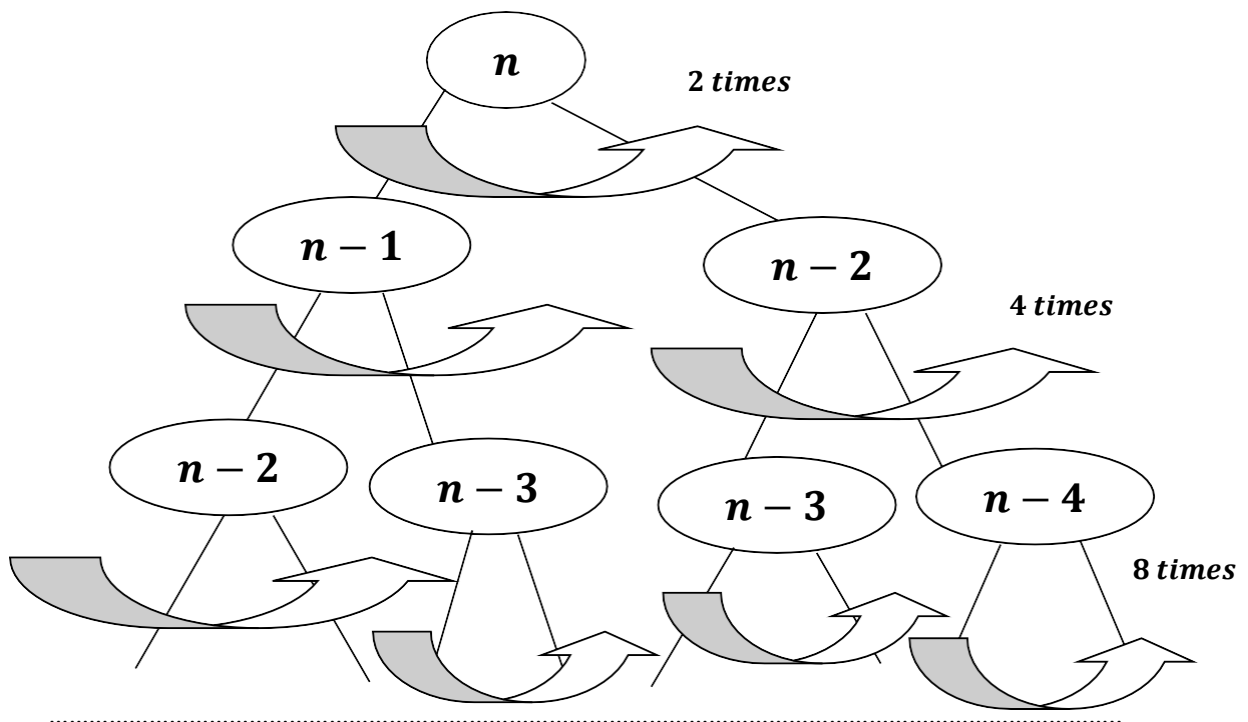
Fibonacci Series Time Complexity with Recursion Tree

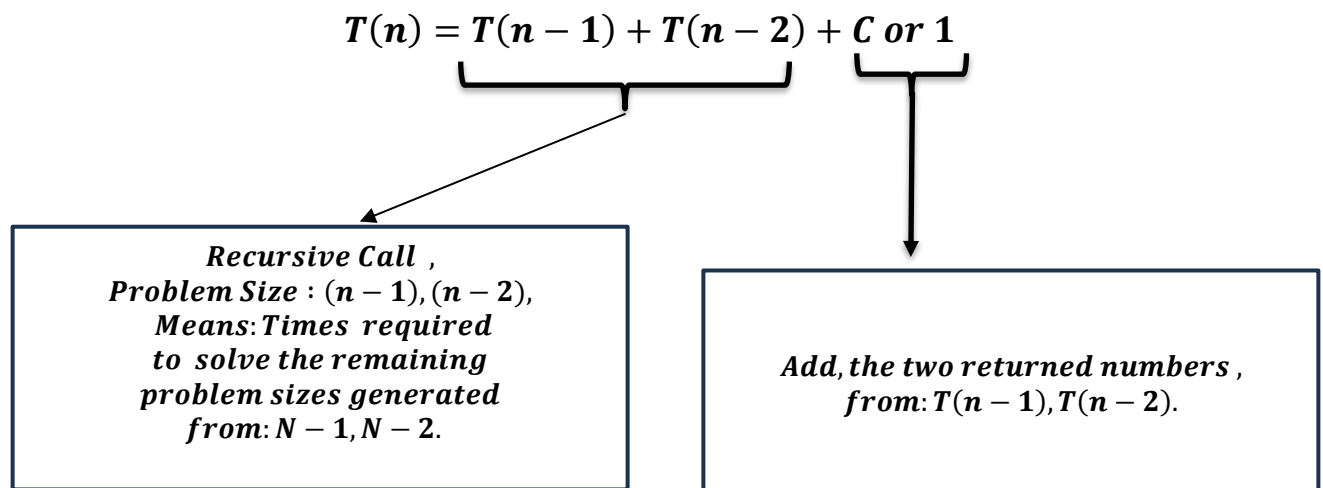
*In fibonacci series : $T(n) = T(n - 1) + T(n - 2)$, where n is greater than 1 ,
when $n = 0$, $T(n) = 0$ and $T(n) = 1$, when $n = 1$, these are base cases for
fibonacci series.*

$$T(n) = \begin{cases} 1 & \text{if}(n = 0) \\ 1 & \text{if}(n = 1) \\ T(n - 1) + T(n - 2) + C & \text{if}(n > 1) \end{cases}$$

To start with: –

If the recursion goes from n to 1 times, considering every nodes, we get:





The Work per Problem: While it waits for those two massive recursive branches to finish, what physical work does the CPU do at the current node? It just adds the two returned numbers together with a +. Because addition is a single basic operation, the work is constant time: $O(1)$, which we will call 'C'.

Level	No. of Problems	Problem Sizes	Work/Cost per Problem	Work done = Work per Problem $\times$ No. of Problems
0	1	n	C	$1 \times C = C$
1	2	$n-1, n-2$	C	$2 \times C = 2C$
2	4	$n-2, n-3,$ $n-3, n-4$	C	$4 \times C = 4C$
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.
k	$\approx 2^k$	(various shrinking sizes)	C	$2^k \times C = 2^k C$

Considering fibonnacci series is a perfect binary tree, then, number of levels:

' k ' = $n - 1$. As $T(1) = 1$ [base case] $\therefore n - k = 1 \Rightarrow k = n - 1$.

Now , we can see growth : $C + 2C + 4C + \dots + 2^k C$.

$$\Rightarrow C(1 + 2 + 4 + \dots + 2^k)$$

$$\Rightarrow C(1 + 2 + 4 + \dots + 2^{n-1}) [k = n - 1]$$

Now we can see the growth = $1 + 2 + 4 + 2^{n-1}$

We can rewrite it as : $2^0 + 2^1 + 2^2 + \dots + 2^{n-1}$

Applying Geometric Progression Series :

$$S(n) = ar^0 + ar^1 + ar^2 + \dots + ar^n = a \left(\frac{r^{n+1} - 1}{r - 1} \right), \text{ where}$$

$S(n)$ is sum of first n terms , a is coefficient and r is common ratio between the consecutive terms.

We get:

$$1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2 + \dots + 1 \times 2^{n-1}$$

$$\Rightarrow 1 \times \left(\frac{2^{n-1+1} - 1}{2 - 1} \right)$$

$$\Rightarrow 2^n - 1$$

$$\therefore C(1 + 2 + 4 + \dots + 2^{n-1})$$

$$\Rightarrow C \times (2^n - 1)$$

Putting Big O , we get :

$$\Rightarrow O(C) \times O(2^n - 1)$$

$$\Rightarrow O(2^n) [As 1 and C are constants].$$
